Project Report

On

HAND GESTURE

Submitted in partial fulfilment of the requirements For the award of the degree of

Bachelor of Technology

In

Computer Science and Engineering

By

Aquib Ali Khan(25SCS1003000491)

S Pawan Das(25SCS1003005177)

Hrishant Jaiswal(25SCS1003003940)

Ayush Anand(25SCS1003005315)

Under the Supervision of

Mr. Anand Mishra

School of Computer Science and Engineering

IILM University

Greater Noida, Uttar Pradesh

CERTIFICATE

This is to certify that the project report entitled "**HAND GESTURE**" submitted by Mr.**Aquib Ali Khan(25SCS1003000491),** Mr.**S Pawan Das(25SCS1003005177),**Mr.**Hrishant Jaiswal(25SCS1003003940),** Mr**.Ayush Anand(25SCS1003005315).**To the IILM University, Greater Noida, Utter Pradesh in partial fulfilment for the award of Degree of Bachelor of Technology in Computer Science & Engineering is a Bonafide record of the minor project work carried out by them under my supervision during the year 2024-2025.

Name (Project Guide)
Designation
School of CSE

Mr.Anand Mishra
Associate Professor Dept. of SCSE

ACKNOWLEDGEMENT

We have taken efforts in this project. However, it would not have been possible without the kind support and help of many individuals and organizations. We would like to extend my sincere thanks to all of them.
We are highly indebted to **Mr.Anand Mishra** for **his** guidance and constant supervision. Also, we are highly thankful to them for providing necessary information regarding the project & also for their support in completing the project.
We are extremely indebted to **Dr. Jasminder Kaur, HOD.** We would also like to express our sincere thanks to all faculty and staff members of School of Computer Science and Engineering, for their support in completing this project on time.
We also express gratitude towards our parents for their kind co-operation and encouragement which helped me in completion of this project. Our thanks and appreciations also go to our friends in developing the project and all the people who have willingly helped me out with their abilities.

(Aquib Ali Khan)

(S Pawan Das)

(Hrishant Jaiswal)

(Ayush Anand)

ABSTRACT

This report explains a practical way to study vegetation using images, especially satellite images. The main goal is to find and highlight areas covered with healthy plants. To do this, a technique called NDVI (Normalised Difference Vegetation Index) is used, which helps measure how healthy and green an area is. First, the satellite images are cleaned and prepared. Then, NDVI is applied to spot the green, plant-covered regions. After that, a process called segmentation is used to separate these green areas from everything else, like buildings or bare ground. To make the results clearer and remove small errors or noise, simple image cleanup steps—called morphological operations like dilation and erosion—are used. These helps sharpen the outlines of the vegetation areas and remove unwanted spots. All of this is done using basic image processing methods that work directly on the images (in the spatial domain), which makes the process fast and compatible with standard remote sensing tools. The final outcome is a reliable map of vegetation that can be used for things like monitoring crops, studying land use, or checking environmental conditions.

CONTENTS

Title	Page
CERTIFICATE	1
ACKNOWLEDGEMENT	2
ABSTRACT	3
CONTENTS	4
CHAPTER 1: INTRODUCTION	5
CHAPTER 2: LITERATURE REVIEW /EXISTING WORK	6
CHAPTER 3: PROBLEM STATEMENT	7
CHAPTER 4: PROPOSED WORK	8
CHAPTER 5: SYSTEM DESIGN [Flow chart, DFD, ERD, Use Case Diagram)	8-10
CHAPTER 6: IMPLEMENTATION (Codes and interface screen shots)	11
CHAPTER 7: CONCLUSION, LIMITATION, AND FUTURE SCOPE	12
REFERENCE	13

1. INTRODUCTION

As technology continues to evolve, the way we interact with computers is also changing. One exciting development is using hand gestures to control devices without touching them. Instead of using a mouse or keyboard, we can now move our hands in front of a camera to perform tasks. This method is especially useful in situations where touchless control is needed—like in virtual reality, video games, smart homes, or for people with physical disabilities.

Hand gesture control works by using a camera to track your hand movements. The computer uses image processing to recognize your hand, understand what gesture you're making, and turn that gesture into a command—like moving the mouse or clicking a button. This project focuses on building a real-time system that lets you control a computer using just your hands and a regular webcam. It uses simple image processing techniques to track gestures and perform actions, making it easy to use, low-cost, and hygienic—since there's no need to touch anything.

Tools and Technologies

To make hand gesture control work, we need both hardware and software working together:

Hardware:

- A webcam or a special depth sensor (like Microsoft Kinect or Intel RealSense) is used to capture hand movements.
- A good graphics card (GPU) helps process video and gestures quickly, especially for real-time applications.

Software and Libraries:

- **OpenCV**: A powerful library that helps with detecting and processing images.
- **MediaPipe**: A tool developed by Google that can track hand positions and finger movements accurately.
- **TensorFlow or PyTorch**: These are used to train AI models that can recognize different hand gestures.
- **Python**: A popular programming language that ties all these tools together.
- **CVZone**: A handy library that makes it easier to combine OpenCV and MediaPipe for faster development.

How It Works

Here's a simple breakdown of how gesture-based control systems work:

- **Detecting the Hand**:
  The camera captures your hand, and software like MediaPipe identifies where your fingers and joints are in real-time.
- **Recognising the Gesture**:
  Simple systems use rules, like "if the thumb is up, it's a thumbs-up gesture."
  Advanced systems use AI models trained to recognise gestures from a set of examples (like recognising letters in sign language).
- **Performing an Action**:
  Once a gesture is recognised, it's linked to a computer action — like moving the mouse, clicking, adjusting volume, or switching apps.

Where It's Used

Hand gesture control has many real-world uses, such as:

- **Touches control** in public places or smart home devices
- **Helping people with disabilities** interact with technology more easily
- **Gaming and virtual reality**, where natural motion adds to the experience
- **Controlling robots or smart devices** without needing to touch anything
- **Giving presentations**, where gestures can replace clickers or remotes

2. LITERATURE REVIEW /EXISTING WORK

Using hand gestures to control computers is a popular research topic in human-computer interaction. Over the years, many methods have been developed to make it easier for people to interact with digital devices using just their hands, without touching anything.

In the beginning, systems used special gloves with sensors (like the DataGlove) to detect hand movements. These systems worked well but were too expensive and not practical for everyday users. As technology improved, researchers started using regular cameras to recognize hand gestures, which made the technology more affordable and easier to use.

One popular tool used in these systems is **OpenCV**, a free software library that helps computers process images. It can detect colors, shapes, and movement in real time. With OpenCV, developers can teach a computer to find a hand in a video and recognize what gesture it is making.

Some researchers have used **machine learning**—especially **Convolutional Neural Networks (CNNs)**—to identify gestures based on images. These advanced methods are very accurate but can be slower and need more computing power. For faster performance in real- time use, many systems use simpler rule-based methods instead.

More recent projects have shown how hand gestures can be used to move the mouse, control music or videos, and even type on a virtual keyboard. Tools like **MediaPipe** by Google have made it even easier by providing ready-made solutions for tracking hands with just a webcam. Overall, while many techniques exist, using a simple webcam and image processing remains the most practical way to recognize hand gestures for daily use.

- **Vision-Based Gesture Recognition**
  In the early stages, hand gesture systems relied on simple cameras and image processing methods like background removal, color detection, and edge finding to recognize hand shapes and movements. While these systems were low-cost, they struggled in poor lighting or with cluttered backgrounds, making them less reliable.
- **Machine Learning Techniques**
  As artificial intelligence advanced, researchers started using machine learning models like SVM (Support Vector Machine), KNN (K-Nearest Neighbors), and CNN (Convolutional Neural Networks) to recognize gestures more accurately. These models are especially good at identifying static hand poses, like showing a number or a specific sign.
- **MediaPipe with Deep Learning**
  Google's MediaPipe changed the game by offering real-time hand tracking using 21 key points (landmarks) on the hand. When combined with deep learning models like CNNs or LSTMs, it becomes possible to recognize both static and dynamic gestures quickly and accurately, even on standard laptops or mobile devices.
- **Rule-BasedSystems**
  Not all systems need complex AI. Some work using simple rules—for example, if the thumb and index finger touch, it's a "pinch" gesture. These systems don't need training data and are easy to build for basic tasks, although they're limited to a small number of gestures.
- **Real-World Applications**
  - At MIT, researchers created a system that lets users control virtual objects just by moving their hands in the air.
  - In India, developers have built low-cost gesture systems using webcams, helping schools and rural areas where expensive hardware isn't available.

- There are also many open-source projects online using tools like OpenCV and Python that let users control the mouse, adjust volume, or skip songs with simple hand movements.

3. PROBLEM STATEMENT

Keyboards and mice are the most common ways to control a computer, but they're not always the best option. In some cases—like for people with physical disabilities, in places where cleanliness is important, or when hands-free control is needed—touching these devices can be difficult or inconvenient.

While there are some systems that let people control computers using hand gestures, many of them are expensive, require special equipment, or are too complicated to work in real-time. This makes them hard to use for everyday purposes.

This project aims to solve that problem by creating a simple and affordable hand gesture control system. It will use just a regular webcam and basic image processing to let users control common computer functions with their hands—no special hardware or complicated software needed. The goal is to make computer interaction easier, touch-free, and accessible to everyone.

Using hand gestures to control a computer is a powerful and modern alternative to traditional devices like the mouse and keyboard. It allows users to interact with the PC in a more natural and touch-free way, which can be especially helpful in situations where hands-free operation is needed — like in medical environments, for people with disabilities, or in smart homes.

However, building a system that can recognize hand gestures accurately and reliably comes with some real challenges:

- **Detecting Gestures Correctly**: The system must work for everyone — no matter their hand size, shape, or skin tone — and must be able to tell different gestures apart.
- **Fast Response**: It needs to work in real-time, with no delays, so users don't feel frustrated or disconnected.
- **Handling Different Environments**: Lighting changes, cluttered backgrounds, or different camera angles can all make it harder for the system to "see" the hand properly.
- **Affordability**: The system should work with basic, low-cost hardware like a regular webcam so that more people can use it.
- **Ease of Use**: It should be simple to learn and use, and flexible enough to support a variety of hand gestures for different tasks.

So, the core challenge is to create a gesture control system that is **accurate, fast, affordable, and easy to use**. Solving this problem will make it possible for more people to interact with computers in a cleaner, more natural, and more accessible way — without ever touching a device.

4. PROPOSED WORK

This project aims to create a system that allows users to control a computer using only hand gestures, with just a regular webcam. The system will use computer vision to understand and respond to hand movements, making it easy and touch-free to interact with the computer.

- **Hand Detection and Tracking**: First, the system will capture video from the webcam. It will use image processing techniques to find and isolate the hand from the rest of the background. The system will then keep track of the hand's position in real-time, making sure it stays detected during use.
- **Gesture Recognition**: Once the hand is detected, the system will recognize specific gestures, such as an open hand, a fist, or pointing. These gestures will be linked to

different computer actions, like moving the mouse, clicking, or scrolling. This will be done using simple methods to extract features from the gestures and possibly some basic algorithms for recognition.

- **Mouse and Control Mapping**: After recognizing the gestures, the system will map them to common computer functions. For example, moving the hand left or right will move the mouse, making a fist will simulate a mouse click, and moving the hand up and down will act like scrolling.
- **Real-time Performance**: The system will be built to work smoothly and quickly in real time, without noticeable delays. The processing will be optimized to ensure the system works well, even in different lighting conditions.
- **User Interface and Calibration**: The system will include a simple interface that lets users adjust the settings, such as the sensitivity of gesture recognition. This will ensure that the system works accurately for different people and in different environments.

5. SYSTEM DESIGN [Flow chart, DFD, ERD, Use Case Diagram)

The system design explains how the hand gesture recognition system works, from capturing input to performing tasks on the computer. It shows the steps involved, how data flows through the system, and how users interact with it. Below are the key parts of the design:

1. Flow Chart:

The flow chart shows the step-by-step process of how the system works. It outlines the entire workflow, from capturing the video to performing the actions.

Flow of the System:

- **Start:** The system starts when it's turned on.
- **Capture Video:** The webcam begins recording video.
- **Detect Hand:** The system identifies the hand using image processing methods like background subtraction.
- **Recognize Gesture:** The system checks the shape or movement of the hand to figure out what gesture is being made (like an open hand, fist, or pointing).
- **Map Gesture to Action:** Once the gesture is recognized, it's connected to a computer action (e.g., move the mouse, click, scroll).
- **Perform Action:** The system executes the action, like moving the mouse or clicking a button.
- **Repeat or End:** The system keeps detecting gestures or can be stopped by the user.

```
+----------------------------+
|        Start        |
+----------------------------+
          | v
+----------------------------+
| Capture Image from Camera|
+----------------------------+
          | v
```

```
+--------------------------------+
| Detect Hand and Landmarks |
+---------------------------------+
          | v
+---------------------------------+
| Recognize Hand Gesture      |
+---------------------------------+
          | v
+---------------------------------+
| Map Gesture to PC Command |
+---------------------------------+
          | v
+---------------------------------+
| Execute Command on PC       |
+---------------------------------+
          | v
+----------------------------+
|         End          |
+----------------------------+
```

2. Data Flow Diagram (DFD):

The DFD shows how the data flows through the system. It illustrates how the hand gestures are processed and how they result in actions on the computer.

DFD Components:

- **External Entity:** The user, who provides gestures through hand movements.
- **Processes:**
  - **Capture Hand Gestures:** The webcam captures the video.
  - **Process Gestures:** The system analyses the video to detect the hand and recognize gestures.
  - **Map Gestures to Actions:** The recognized gestures are linked to specific computer actions.
- **Data Storage:** The system might save gesture preferences, like sensitivity settings.
- **Output:** The system sends the computer action (e.g., moving the mouse or clicking) as the output.

```
+--------------------+
|     User       |
+--------------------+
      |
+-------+-------+
```

```
        |          |
[Capture Hand Gesture]
            |
[Detect Hand Landmarks]
            |
[Recognize Gesture]
            |
[Map Gesture to Command]
            |
[Execute PC Command]
```

3. Entity-Relationship Diagram (ERD):

The ERD shows how different components of the system are related. Even though the system doesn't need a complex database, this diagram helps explain how the gestures, actions, and user settings interact.

ERD Components:

- **Entities:**
  - Gestures (e.g., open hand, fist, pointing)
  - Actions (e.g., move mouse, click, scroll)
  - User Preferences (e.g., gesture sensitivity)
- **Relationships:**
  - Gestures are connected to specific Actions (e.g., a fist gesture might map to a mouse click).
  - User Preferences can adjust the sensitivity of Gestures for personalization.

4. Use Case Diagram:

The use case diagram shows how the user interacts with the system. It illustrates what actions the user can take and how the system responds.

Use Case Components:

- **Actor:** The user, who interacts with the system by performing hand gestures.
- **Use Cases:**
  - **Perform Gesture:** The user makes a hand gesture in front of the webcam.
  - **Recognize Gesture:** The system processes the gesture.
  - **Execute Action:** The system performs the corresponding action on the computer, like moving the mouse or clicking.
  - **Customize Settings:** The user can adjust settings like gesture sensitivity or preferences for better accuracy.

```
[User] ------< performs >------ [Gesture] ------< contains > ------[Hand_Landmark]
  |                            |
  |                            v
  |--------------------> [Command] < -----------[System_Log]
```

6. IMPLEMENTATION (Codes and interface screen shots)

This section explains how the hand gesture recognition system works. It describes the key components: capturing video from the webcam, detecting hand gestures, mapping those gestures to actions, and then performing those actions on the computer.

1. Code Overview:

The system is built using Python and the OpenCV library to process video in real time. Here's a basic breakdown of the main steps:

- **Capture Video Input**: The webcam continuously captures live video frames, which are processed immediately by the system.

Python code

```
cap = cv2.VideoCapture(0)
```

- **Hand Detection**: The system uses techniques like background subtraction and skin color filtering to detect the hand in the video feed.

Python code

```
hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
lower_skin = np.array([0, 20, 70], dtype=np.uint8)
upper_skin = np.array([20, 255, 255], dtype=np.uint8)
mask = cv2.inRange(hsv, lower_skin, upper_skin)
```

- **Gesture Recognition**: After detecting the hand, the system identifies the gesture (such as an open hand or a fist) and links it to a computer action like moving the mouse or clicking.

Python code

```
if hand_detected():
   if is_fist(gesture):
      simulate_click()
   elif is_pointing(gesture):
      move_mouse()
```

- **Mouse and Control Mapping**: The system then converts the recognized gestures into actions on the computer, like moving the mouse or clicking a button. This is done using a library called pyautogui.

Python code

```
pyautogui.moveTo(x, y)
pyautogui.click()
```

2. Interface Screenshots:

- **Live Video Feed**: A screenshot of the live webcam feed where the system detects the hand.
- **Gesture Recognition Display**: A screenshot showing the system recognizing the hand gesture in real-time, and highlighting the corresponding action (like a mouse click or movement).
- **Settings Interface**: A simple settings screen where users can adjust the sensitivity of gesture recognition and other parameters for better performance.

7. CONCLUSION, LIMITATION, AND FUTURE SCOPE

Conclusion:

The hand gesture recognition system created in this project effectively allows users to control a computer using just a standard webcam, without the need for touch-based input devices. By applying computer vision techniques and Python libraries like OpenCV, the system can detect hand movements in real-time and map them to actions like moving the mouse, clicking, and scrolling. This system provides a simple, user-friendly, and efficient alternative to traditional input devices, especially in situations where touch-free control is needed, such as for hygiene or accessibility. It is cost-effective and provides a reliable way to interact with digital devices.

Limitations:

While the system works well, it has a few limitations:

- **Lighting Sensitivity**: The system's performance can decrease in poor or uneven lighting, which might cause the hand gestures to be recognized inaccurately.
- **Limited Gesture Variety**: Currently, the system can only recognize a few basic gestures (like an open hand or fist). The system could be improved by adding more complex gestures for better control.
- **Environmental Factors**: Things like background clutter or moving objects can interfere with the system's ability to detect hand gestures accurately.
- **Computational Resources**: The system runs on regular hardware, but performance might vary depending on the computer's processing power, particularly when handling multiple gestures at once.

Future Scope:

There are many ways the system can be improved:

- **Multi-Gesture Recognition**: Future versions could recognize a wider range of gestures, such as finger counting or swiping, for more control options.
- **Integration with Other Devices**: The system could work with other smart devices, like smart TVs or home automation systems, for a more connected control experience.
- **Improved Machine Learning**: Using advanced machine learning models could improve the accuracy of gesture recognition, especially in challenging conditions like low light or busy backgrounds.
- **Real-Time Calibration**: The system could allow users to adjust sensitivity and other settings in real-time to suit their needs, making it more adaptable.
- **Cross-Platform Compatibility**: Expanding the system to work on different operating systems (Windows, macOS, Linux) and with various applications would make it more versatile and accessible.

REFERENCE:

- **MediaPipe Documentation:** Offers detailed information about hand tracking, pose detection, and API usage. https://google.github.io/mediapipe/
- **OpenCV Official Site:** Provides resources, tutorials, and guides for using OpenCV in real-time applications. https://opencv.org/
- **PyAutoGUI Documentation:** Discusses various automation features and examples for triggering system-level actions. https://pyautogui.readthedocs.io/
- **Python Programming Language:** Ensures compatibility with Python 3.x. https://www.python.org/